

Laboratorium 8: Sieci konwolucyjne – analiza sygnałów audio

Zadanie 1: Pobranie i wstępne przetwarzanie zbioru danych

Cel: Przygotowanie zbioru danych do klasyfikacji sygnałów audio słów

TensorFlow opublikował Speech Commands Datasets. Baza danych zawiera 65 tysięcy jednosekundowych nagrań 30 krótkich słów w języku angielskim, wypowiedzianych przez kilka tysięcy różnych osób. Zbiór danych należy przygotować przed wykonaniem zadania klasyfikacji wypowiedzi.

1. Dane można ściągnąć z tej lokalizacji: http://download.tensorflow.org/data/speech_commands_v0.01.tar.gz

2. Dodatkowe biblioteki, które będą przydatne przy przetwarzaniu danych:

```
librosa
IPython (IPython.display)
scipy (from scipy.io import wavfile)
```

3. Wizualizacja przykładowego sygnału audio

```
train_audio_path = './input/'
samples, sample_rate = librosa.load(train_audio_path+'yes/0a7c2a8d_nohash_0.wav', sr = 16000)

fig = plt.figure(figsize=(14, 8))
ax1 = fig.add_subplot(211)
ax1.set_title('Raw wave of ' + './input/0a7c2a8d_nohash_0.wav')
ax1.set_xlabel('time')
ax1.set_ylabel('Amplitude')
ax1.plot(np.linspace(0, sample_rate/len(samples), sample_rate), samples)
```

4. Odsłuchanie przykładowego sygnału audio w notatniku (po wyświetleniu kontrolki "play"):

```
import IPython.display as ipd
ipd.Audio(samples, rate=sample_rate)
```

5. Funkcja librosa.load zwraca informację o częstotliwości próbkowania sygnału audio. Z informacji zwróconej przez funkcję load wynika, że częstotliwość próbkowania nagrań to 16000 Hz. Aby skrócić czas treningu klasyfikatorów należy wykonać resamplig sygnału usuwając z sygnału częstotliwości powyżej 8000 Hz. Spowoduje to oczywiście pogorszenie jakości słyszanego dźwięku, ponieważ częstotliwości słyszalne przez ludzkie ucho mieszczą się w przedziale do ok. 22 000 Hz.

```
samples = librosa.resample(np.asarray(samples, dtype=np.float32), orig_sr=sample_rate, target_sr=8000)
ipd.Audio(samples, rate=8000)
```

6. Z bazy nagrań należy wybrać dowolne dziesięć słów, a następnie należy:

- sprawdzić ile jest w bazie danych nagrań dla każdego wybranego polecenia głosowego
- sprawdzić, jak długie są nagrania poleceń głosowych
- wykonać resampling nagrań do częstotliwości 8 kHz i pozostawić tylko nagrania jednosekundowe (złożone z 8000 próbek)
- każdą przetworzoną próbkę należy zapisać na dysku

7. W oparciu o przetworzone dane należy przygotować implementację własnego datasetu (przyjmujący w konstruktorze listę lokalizacji plików, etykiety mają być dedukowane z nazwy kartoteki, w której umieszczono przetworzone nagranie) i datalodera (proszę konsekwentnie używać ważonego próbkowania).

Zadanie: Przygotuj dane do treningu klasyfikatora opartego o głęboką sieć konwolucyjną.

Zadanie 2: Sieć konwolucyjna do klasyfikacji nagrań

Cel: Implementacja i trening sieci konwolucyjnej do klasyfikacji wybranych nagrań

1. Studenci implementują i sieć konwolucyjną według schematu:

Layer (type)	Output Shape
input_1 (InputLayer)	[(None, 8000, 1)]
conv1d (Conv1D)	(None, 7988, 8)
max_pooling1d (MaxPooling1D)	(None, 2662, 8)
dropout (Dropout)	(None, 2662, 8)
conv1d_1 (Conv1D)	(None, 2652, 16)
max_pooling1d_1 (MaxPooling1D)	(None, 884, 16)
dropout_1 (Dropout)	(None, 884, 16)
conv1d_2 (Conv1D)	(None, 876, 32)
max_pooling1d_2 (MaxPooling1D)	(None, 292, 32)
dropout_2 (Dropout)	(None, 292, 32)
conv1d_3 (Conv1D)	(None, 286, 64)
max_pooling1d_3 (MaxPooling1D)	(None, 95, 64)
dropout_3 (Dropout)	(None, 95, 64)
flatten (Flatten)	(None, 6080)
dense (Dense)	(None, 256)
dropout_4 (Dropout)	(None, 256)
dense_1 (Dense)	(None, 128)
dropout_5 (Dropout)	(None, 128)
dense_2 (Dense)	(None, 10)

Dodatkowe wskazówki do schematu sieci:

- warstwy MaxPooling operują na oknie o szerokości 3
- liczba kanałów w kolejnych warstwach konwolucyjnych: 8, 16, 32, 64
- rozmiar kerneli w kolejnych warstwach konwolucyjnych: 13, 11, 9, 7

- dla warstw konwolucyjnych użyto argumentu padding='valid'
- dropout rate: 0.3
- aktywacje ReLU po każdej warstwie konwolucyjnej (przed MaxPooling)
- liczba wyjść (10) równa liczbie klas

- # 2. Przygotowane dane należy podzielić na część treningowo-walidacyjną i testową
- # 3. Trening sieci w schemacie walidacji krzyżowej pięciokrotnej z logowaniem do tensorboard
- # 4. Wykonanie testów na zbiorze treningowym. Wyznaczenie macierzy pomyłek, raport z klasyfikacji - w oparciu o predykcję "ensemblową".
- # 5. Wykonanie testów na zbiorze testowym. Wyznaczenie macierzy pomyłek, raport z klasyfikacji - w oparciu o predykcję "ensemblową". Ocena niepewności predykcji: identyfikacja przypadków o największej niepewności predykcji.
- # 6. Porównanie wyników testów dla zbiorów treningowych i testowych – ocena przeuczenia.

Zadanie: Zaimplementuj i wytrenuj sieć do klasyfikacji poleceń głosowych.

Zadanie 3: Wybór hiperparametrów

Cel: Optymalizacja hiperparametrów sieci konwolucyjnej

- # Studenci implementują kod do optymalizacji hiperparametrów sieci:
- # 1. Dla ustalonego batch size (np. 32) należy przygotować kodu do optymalizacji:
 - learning rate
 - weight decay
 - dropout rate
- # 2. Wykonanie optymalizacji w optunie (logowanie wyników z triali do tensorboard)
- # 3. Trening w oparciu o optymalny zestaw hiperparametrów z wykorzystaniem walidacji krzyżowej pięciokrotnej (logowanie do tensorboard)
- # 4. Testy na zbiorze treningowym i testowym wg. schematu jak w Zadaniu 2

Zadanie: Wykonaj optymalizację hiperparametrów treningu sieci z Zadania 2

Zadanie 4: Wpływ liczebności zbioru treningowego na dokładność klasyfikacji w zbiorze testowym

Cel: Ocena wpływu liczebności zbioru testowego na dokładność klasyfikacji

- # Studenci trenują sieć z Zadania 3:
- # 1. dla 20% losowo wybranych próbek ze zbioru treningowego (oczywiście należy użyć stratyfikacji)
- # 2. dla 40%, 60%, 80% losowo wybranych próbek ze zbioru treningowego
- # 3. wytrenowane ensemble (20%, 40%, 60%, 80%, 100% z zadania 3) należy użyć do wykonania testów na zbiorze testowym.
- # 4. należy wykonać wykres przedstawiający zależność dokładności klasyfikacji od liczebności zbioru treningowego

Zadanie: Znajdź „optymalną” wielkość zbioru treningowego – taką wielkość zbioru treningowego, że dalsze jego powiększanie nie niesie realnego zysku w postaci istotnie lepszej klasyfikacji.

Zadanie 5: Transfer learning

Cel: Wykorzystanie transfer learning w treningu klasyfikatora poleceń głosowych

- # Z bazy nagrań należy wybrać **pięć** poleceń innych niż użyte w zadaniach 1- 4
- # 1. Dane należy przeprocесować zgodnie ze schematem z zadania 1
- # 2. W oparciu o wyniki Zadania 4, dla każdej z pięciu klas należy wybrać losowo liczbę próbek, która nie zapewnia zadowalających wyników treningu
- # 3. Przygotowanie sieć wg. schematu z zadania 2, podmieniając ostatnią warstwę liniową na warstwę z **pięcioma** wyjściami
- # 4. Trening modelu bazowego – sieć z pkt. 3 należy wytrenować „from scratch”, korzystając z zestawu hiperparametrów z zadania 3, jak zwykle trening w schemacie walidacji krzyżowej pięciokrotnej
- # 5. Transfer learning – warstwy konwolucyjne sieci z pkt. 3 należy zainicjować wagami modeli z zadania 3, warstwy te należy zamrozić (“requires gradient” ustawione na False), a następnie model należy dotrenować wybierając optymalną optymalne learning rate i weight decay
- # 6. Testy i porównanie (statystyczne) wytrenowanych modeli na zbiorze testowym

Zadanie: W oparciu o pretrenowaną sieć wytrenuj klasyfikator do rozpoznawania nowego zestawu poleceń głosowych.

Zadanie 6: Augmentacja danych

Cel: Uzupełnienie kodu dataset o transformacje wykonywane „on the fly”

- # Studenci implementują:
- # 1. metodę augmentacji sygnału audio, polegającą na dodaniu do sygnału szumu gaussowskiego; metodę należy przetestować na kilku nagraniach, oceniając słuchowo wpływ augmentacji na jakość sygnału
- # 2. metodę augmentacji polegającą na zmianie wysokości głosu osoby wypowiadającej – efekt można zrealizować rozciągając/ kurcząc w czasie nagranie; metodę również należy przetestować słuchowo
- # 3. inne metody augmentacji z bibliotek librosa i torchaudio
- # 4. dataset z transformacjami stosowanymi „on the fly” – listę transformacji należy przekezać, jako argument konstruktora datasetu
- # 5. trening modelu z zadania 3 z wykorzystaniem zmodyfikowanego datasetu
- # 6. testy na zbiorze testowym i porównanie (statystyczne) wyników dla treningu z i bez (model z zadania 3) augmentacji

Zadanie: W oparciu o dataset uzupełniony o metody augmentacji należy wytrenować i przetestować sieć do klasyfikacji nagrań głosowych.